Лабораторна робота №5

Мета роботи: Сформувати практичні навички запису команд руху робота, їх збереження у файл та подальшого відтворення, а також зрозуміти принципи реалізації сценарної поведінки робототехнічної системи.

Посилання на репозиторій: [Anastasia11070830/ROS2]

Хід роботи

1. Використовуємо ту саму віртуальну систему що і раніше.
2. **Підготовка середовища**

Робимо активацію середовища та робимо перевірку топіка /cmd_vel Топік /cmd_vel з'являється, коли хтось на нього підписаний, тож спершу запускаємо симулятор і Motion Node.

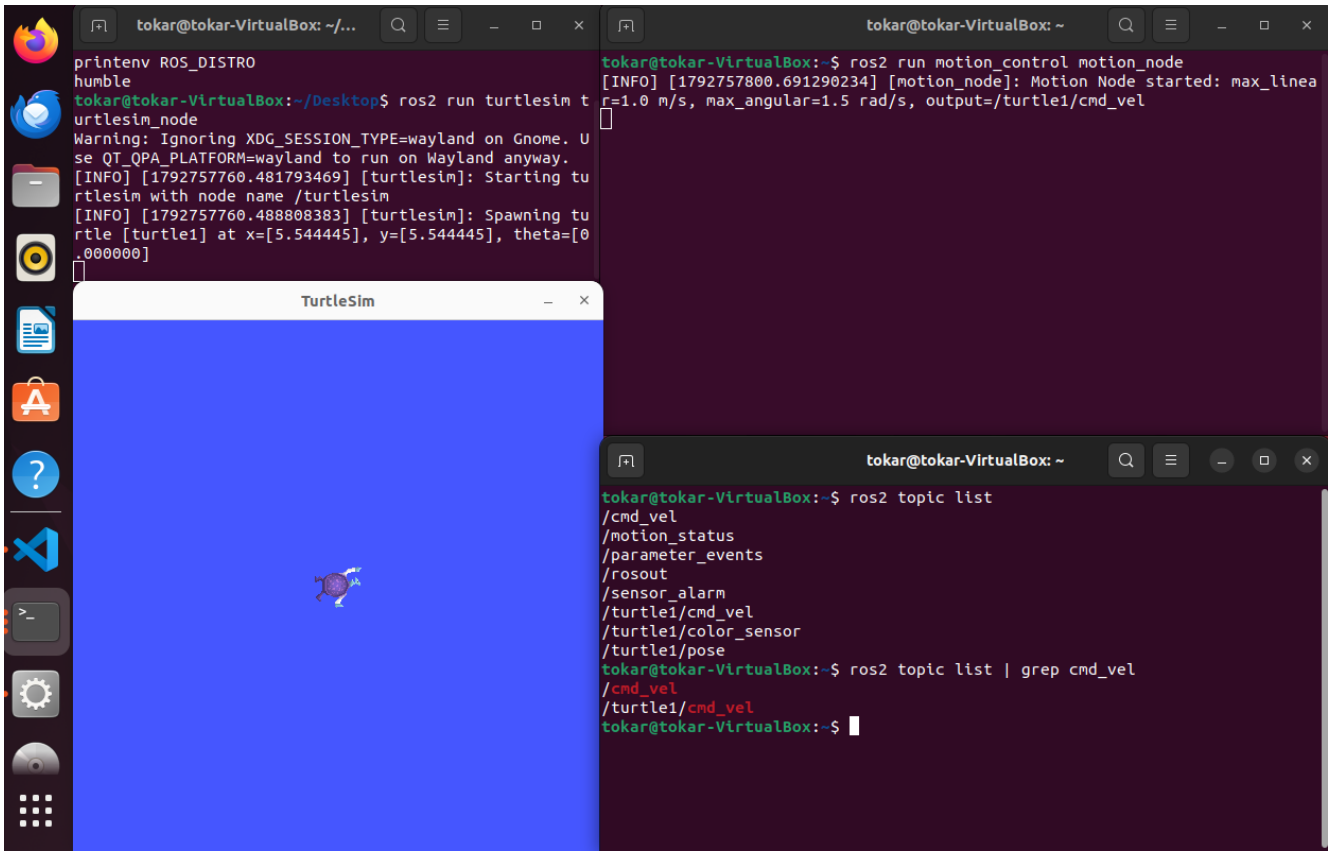


Рисунок 1 – Підготовка середовища

3. **Створення пакету Record/Playback**

На даном етапі ми робимо каркас для нової програми, яка буде слухати команди руху (завдяки geometry_msgs), запам'ятовувати їх і потім відтворювати.

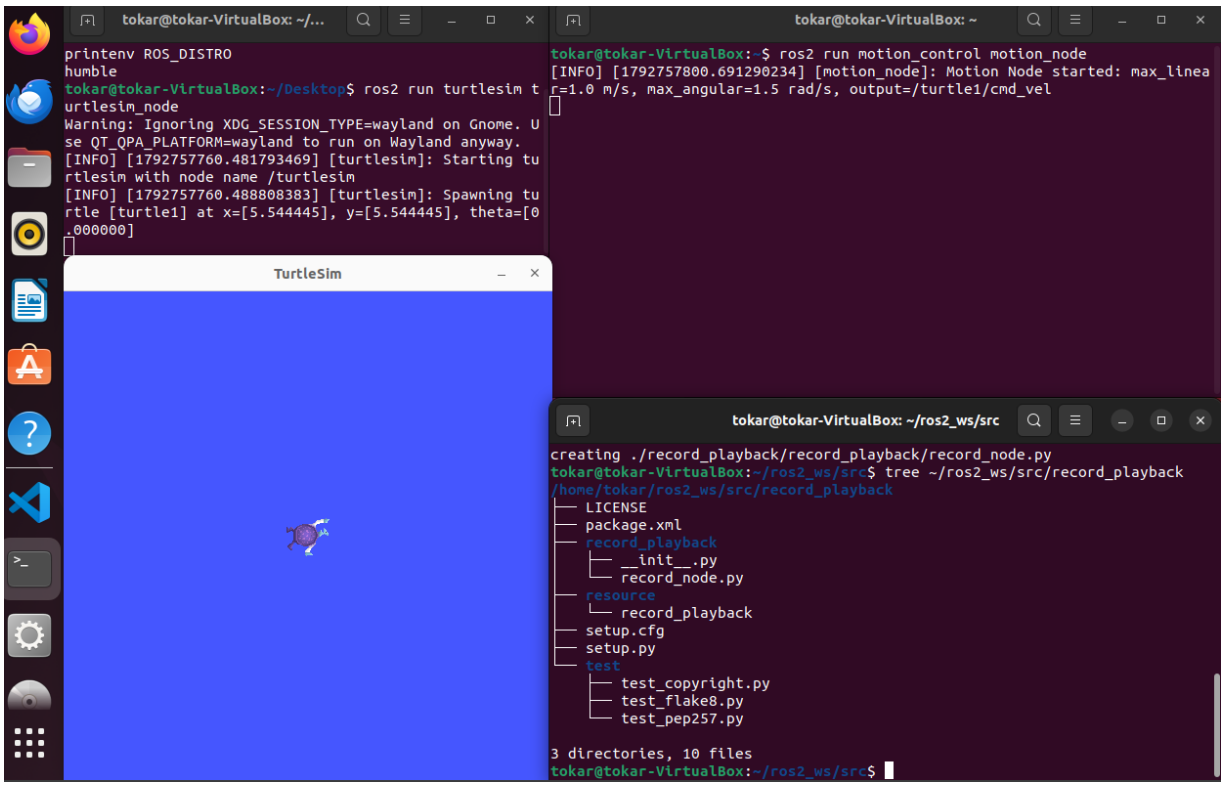


Рисунок 2 – Процес створення

4. Record Node: підписка на /cmd_vel і запис у файл

Відкриваємо код у VS code. Пишемо код для нового вузла (Node), який буде записувати команди руху робота у файл. Це проміжний етап створення системи, яка спочатку запам'ятає траєкторію (або команди керування), а потім зможе її відтворити.

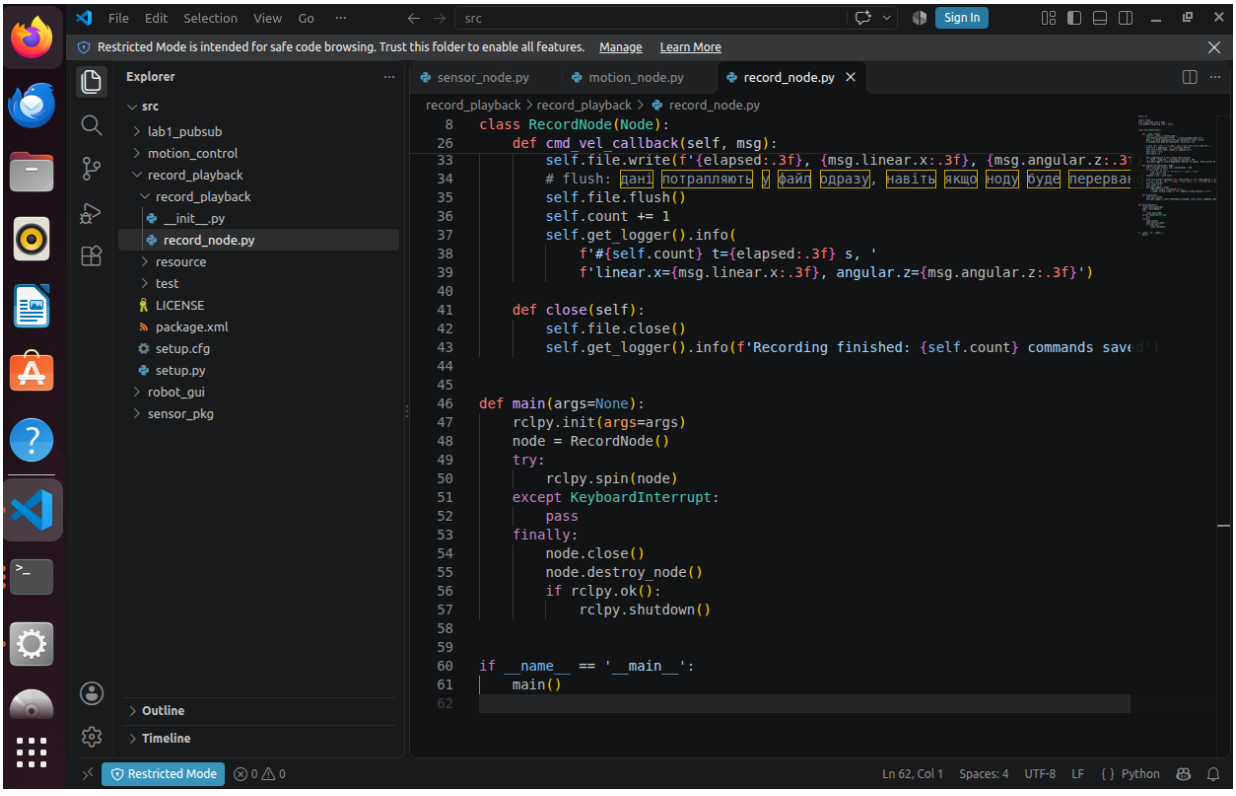
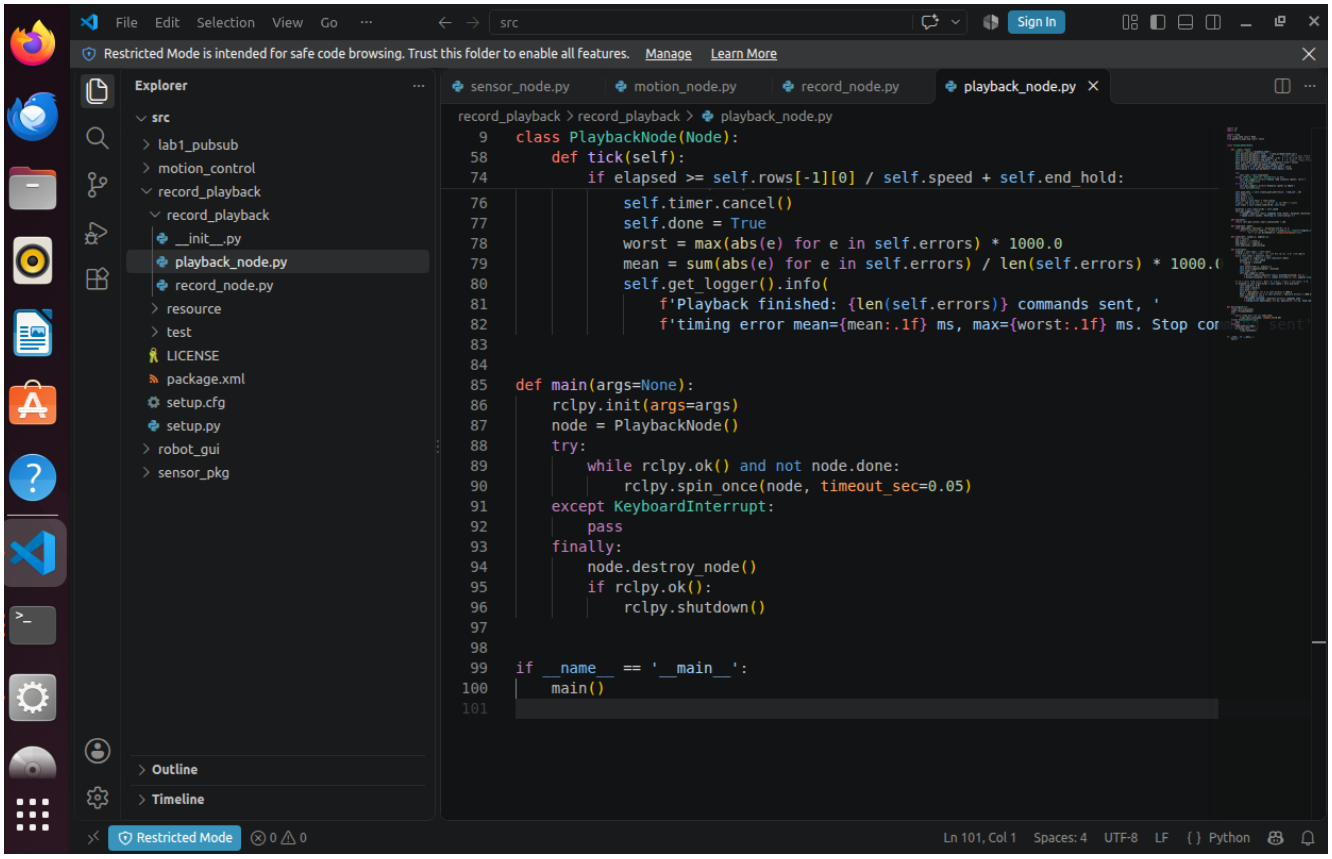


Рисунок 3 – Record code

5. Код Playback Node

На даному етапі був зроблений скрипт, який перетворює статичний CSV-файл із записаними даними назад у реальні команди для робота, зберігаючи початкові таймінги (або змінюючи їх за потреби) і коректно завершуючи свою роботу після досягнення кінця файлу.




```python
class PlaybackNode(Node):
    def tick(self):
        if elapsed >= self.rows[-1][0] / self.speed + self.end_hold:
            self.timer.cancel()
            self.done = True
            worst = max(abs(e) for e in self.errors) * 1000.0
            mean = sum(abs(e) for e in self.errors) / len(self.errors) * 1000.(
            self.get_logger().info(
                f'Playback finished: {len(self.errors)} commands sent, '
                f'timing error mean={mean:.1f} ms, max={worst:.1f} ms. Stop cor

def main(args=None):
    rclpy.init(args=args)
    node = PlaybackNode()
    try:
        while rclpy.ok() and not node.done:
            rclpy.spin_once(node, timeout_sec=0.05)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Рисунок 4 – Playback node

6. Перевірка package.xml

На даному ертапі відбувається перевірка та налаштування конфігураційних файлів пакету record_playback перед його фінальною збіркою.Це важливий етап, щоб переконатися, що система ROS 2 знає, які бібліотеки використовує пакет і які програми (вузли) вона має вміти запускати.

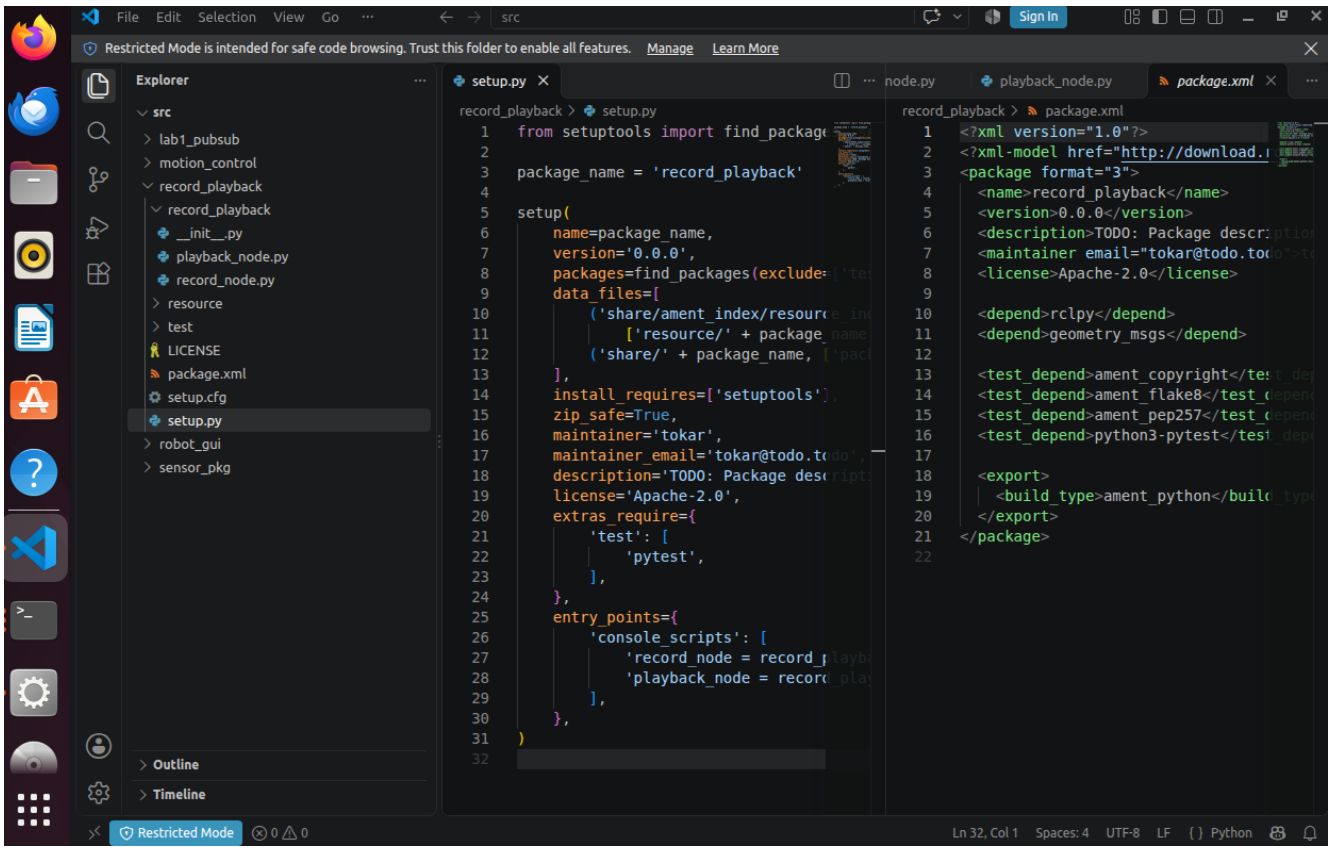


Рисунок 5 - package.xml

7. Збірка

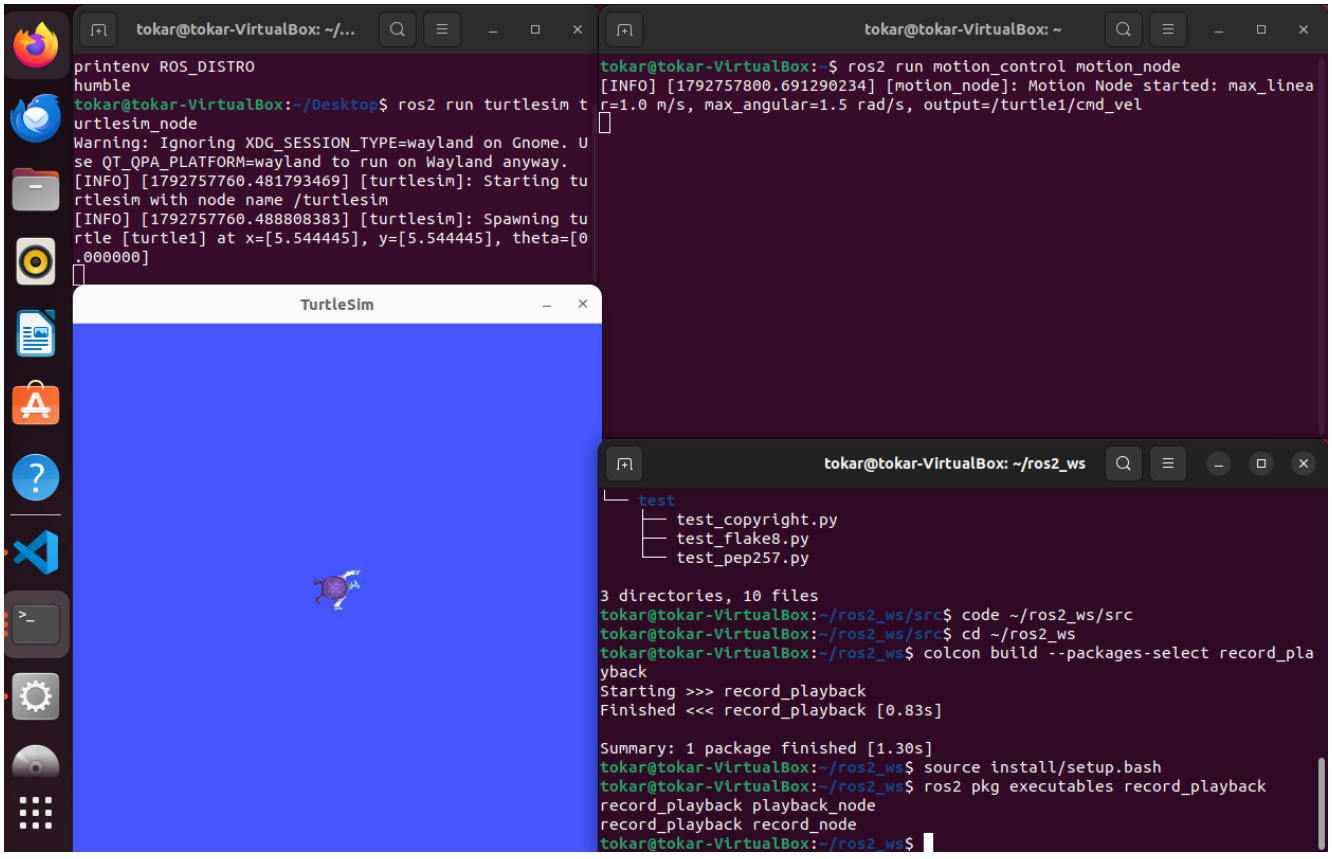


Рисунок 6 – Збірка

8. Вводимо дані

Вводимо Лінійна швидкість = 0.50, поле Кутова швидкість = 1.0. Робот почав рухатися по колу, а програма-реєстратор (record_node) у цей момент сумлінно записує ці швидкості у файл, щоб потім вузол playback_node зміг відтворити цей самий шлях.

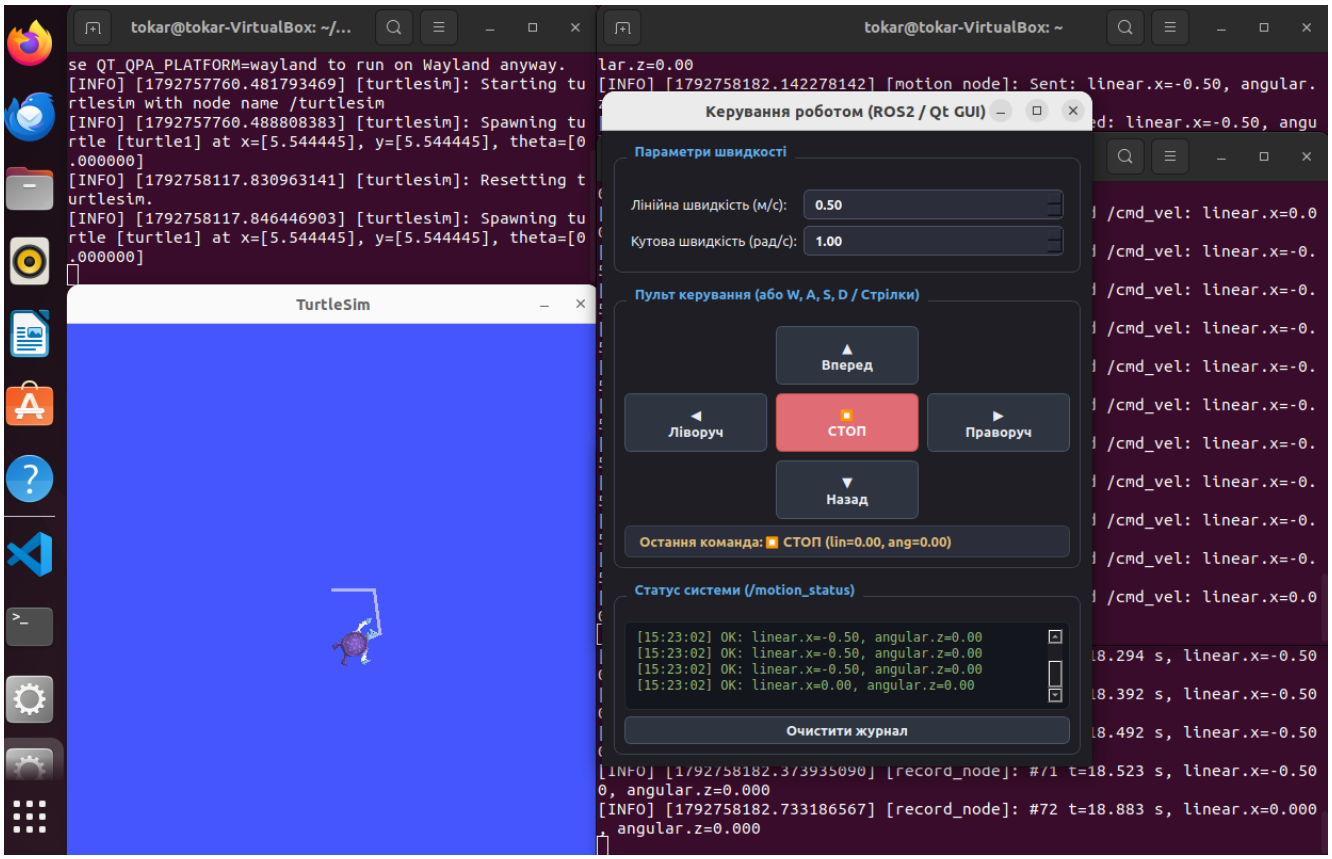


Рисунок 7 – демонмтрація ведених даних

9. Перегляд файлу сценарію

Файл route.csv містить реальні команди, які ви відправляли, із точними часовими інтервалами. Це означає, що етап запису (Record) пройшов успішно, і тепер цей файл можна згодовувати вузлу playback_node, щоб черепаха самостійно повторила цей шлях.

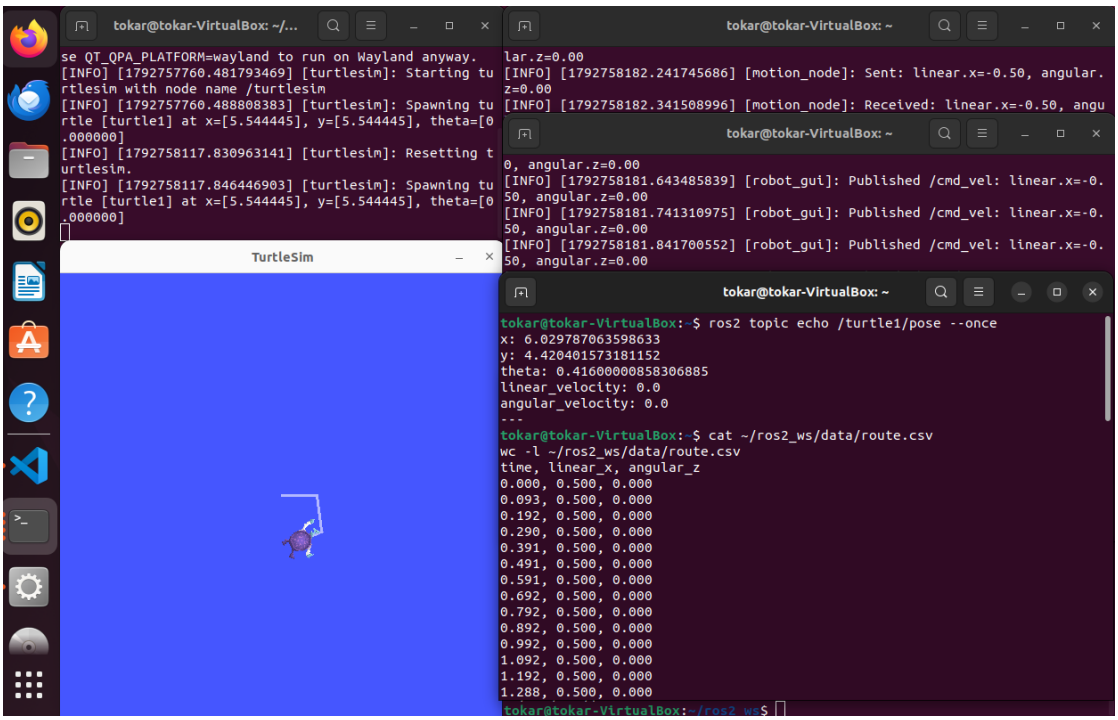


Рисунок 8 – перегляд файлу

10. Відтворення сценарію

Система здатна не лише записувати команди керування роботом у файл, а й з високою точністю відтворювати їх, змушуючи робота автономно повторювати заданий сценарій.

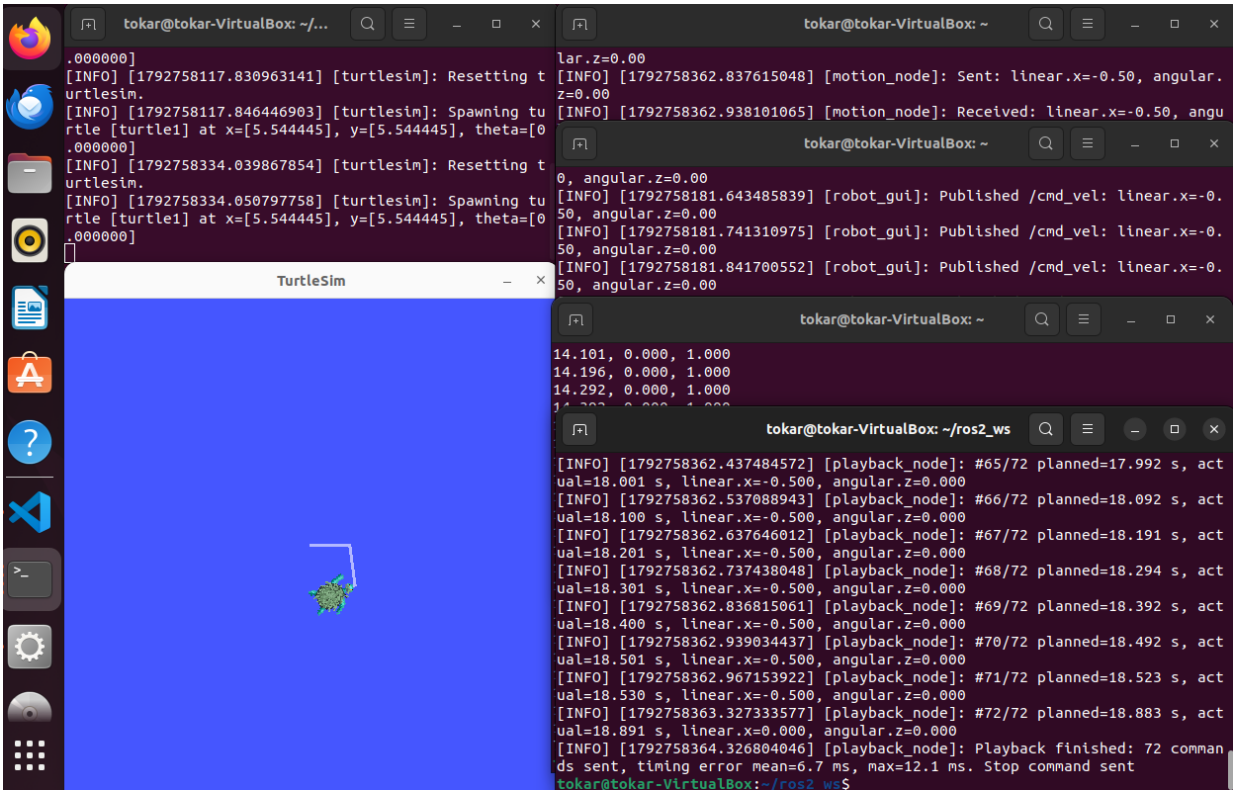


Рисунок 9 – відтворення

11.Порівняння кінцевої позиції

Довели на даному етапі що система може не лише візуально, а й з високою точністю повторювати траєкторію руху робота.

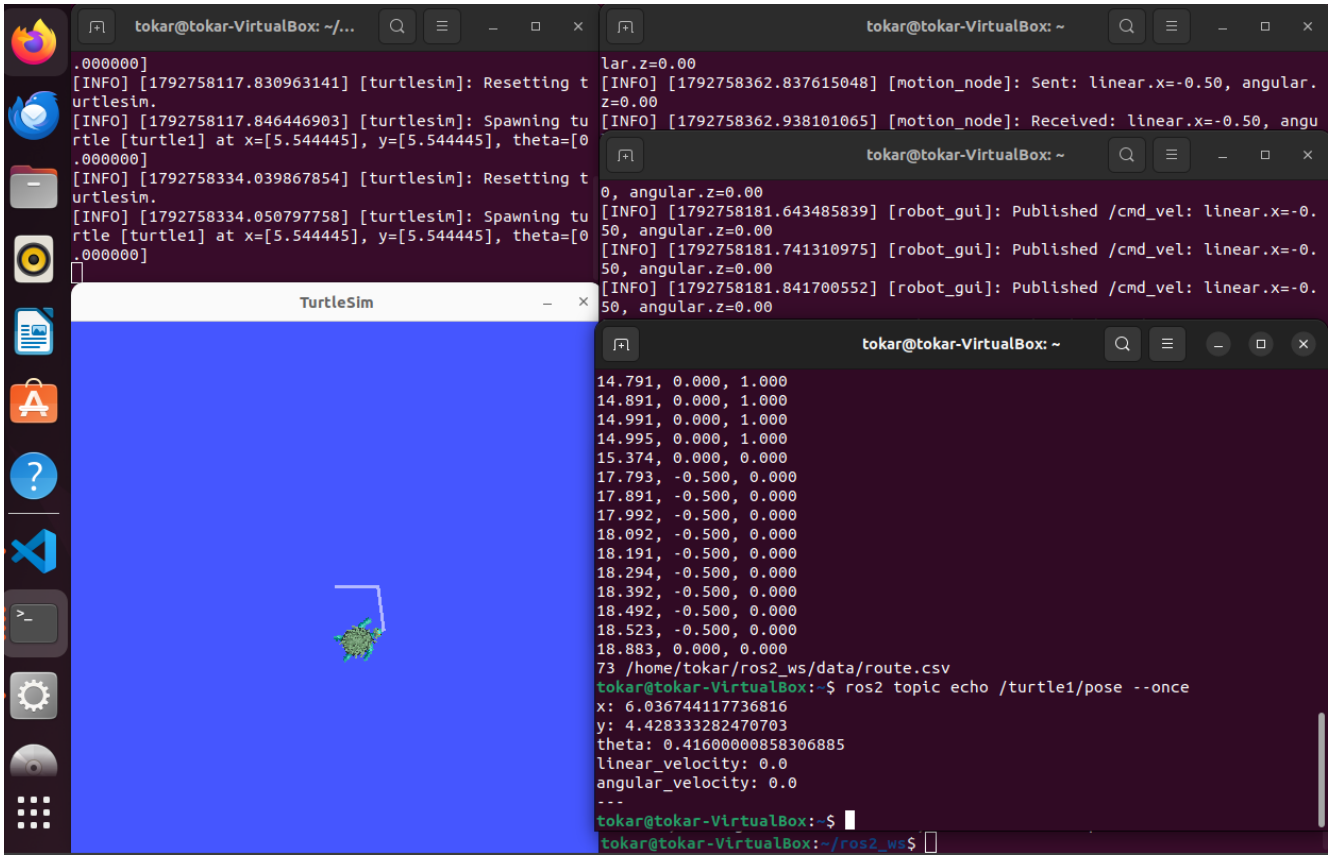


Рисунок 10 – Порівняння

12.Об'єктивна перевірка: «запис запису» (послідовність і затримки)
Поки сценарій відтворюється, другий record_node записує те, що надсилає playback_node. Далі порівнюємо вихідний файл і запис відтворення.

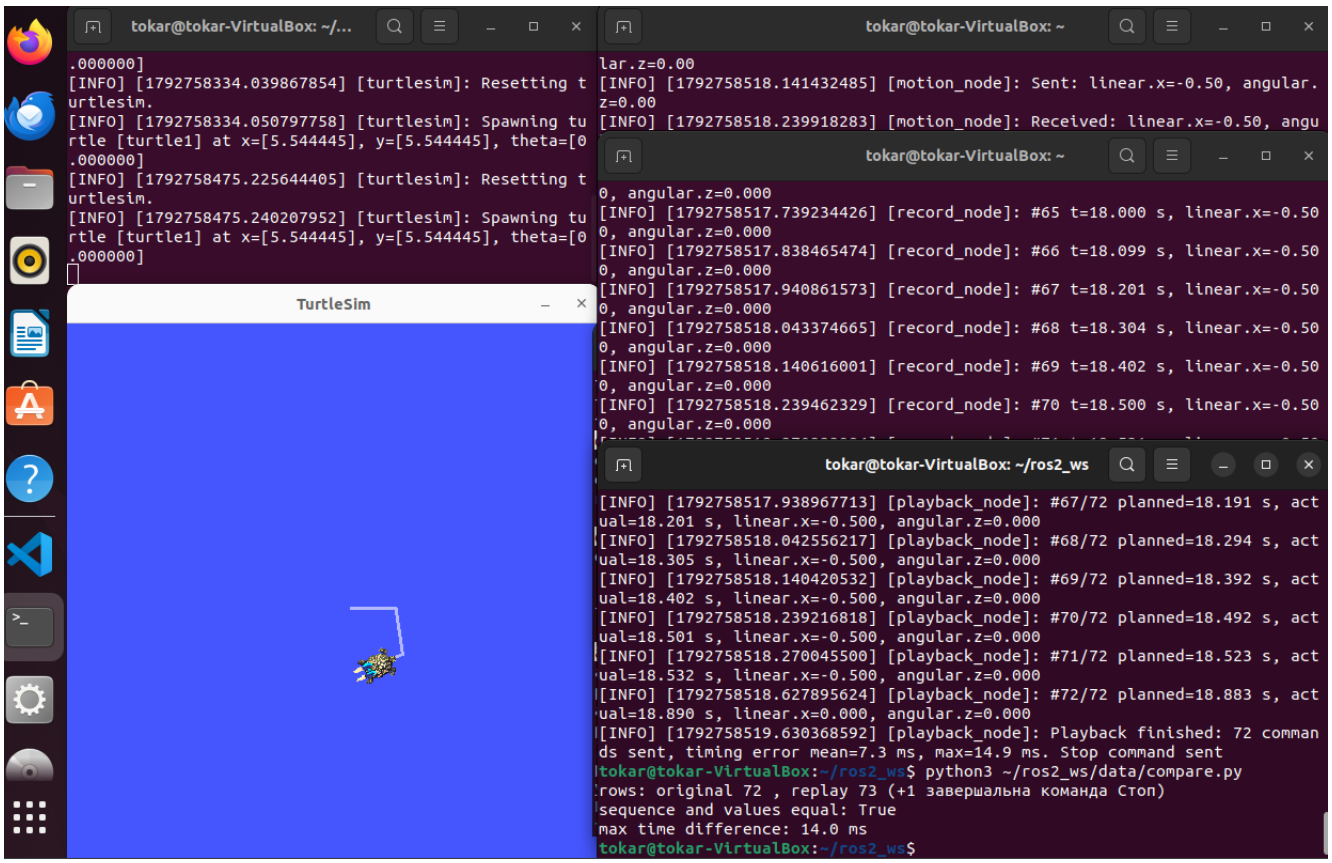


Рисунок 11 – Порівняння

Висновок: У ході виконання лабораторної роботи було успішно досліджено архітектуру та принципи взаємодії вузлів у ROS 2. Спочатку було створено пакет сенсора та реалізовано систему безпеки, яка через топіки передає сигнал тривоги контролеру руху, динамічно блокуючи небезпечний поступальний рух при зміні параметрів "на льоту". Далі розроблено пакет для запису та відтворення сценаріїв, що дозволяє зберігати команди керування у CSV-файл і згодом автономно їх відтворювати. За допомогою автоматизованого скрипта було доведено високу точність роботи системи: відтворена послідовність команд повністю збігається з оригіналом, а похибка часу становить лічені мілісекунди. Отримані навички демонструють ефективність використання ROS 2 для створення модульних, надійних та інтегрованих робототехнічних систем.